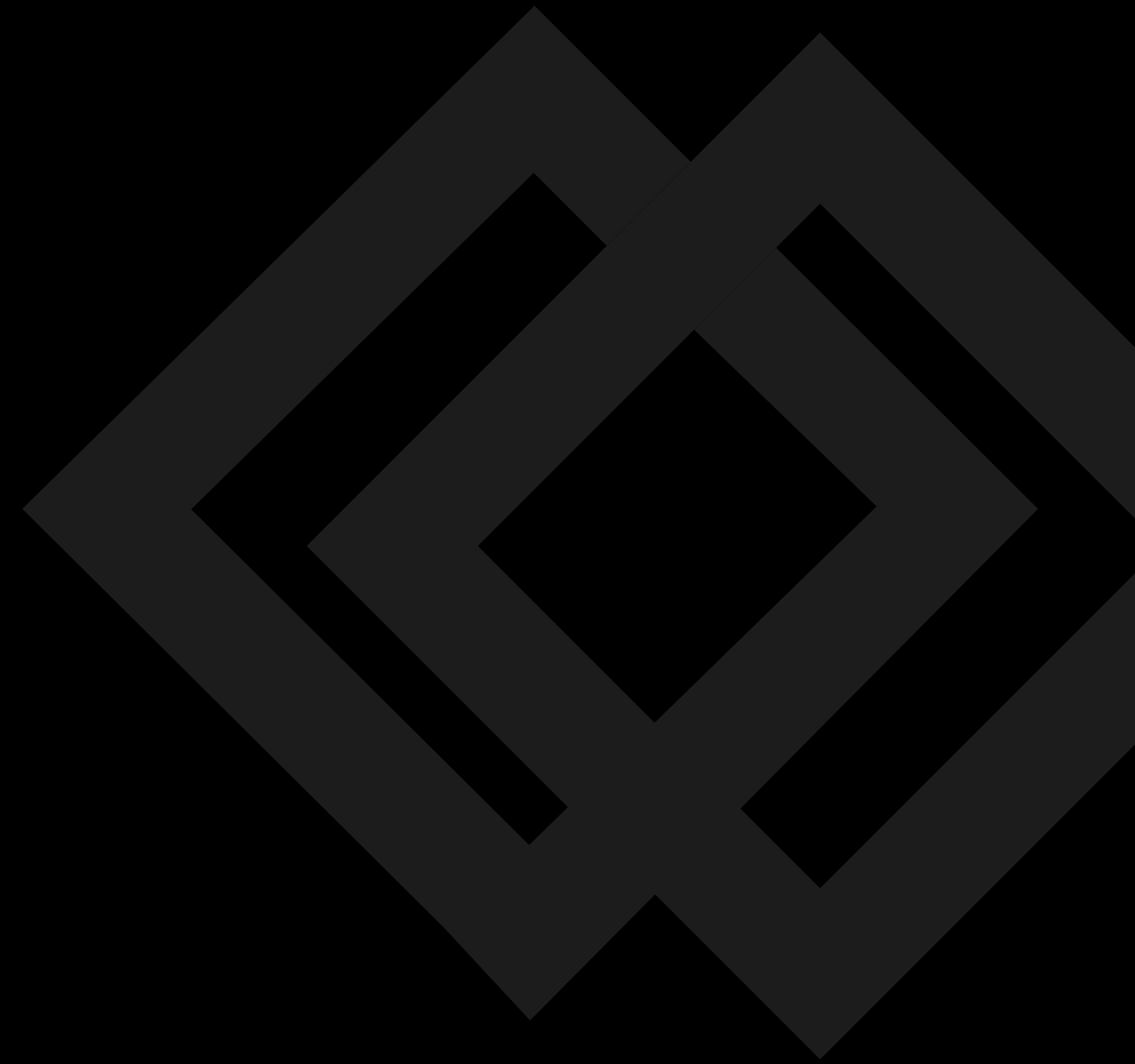


탄막 게임



97기 허태웅

목차

1

게임 설명

2

주요 기능

3

보완할 점

1. 게임설명

1. 모티브

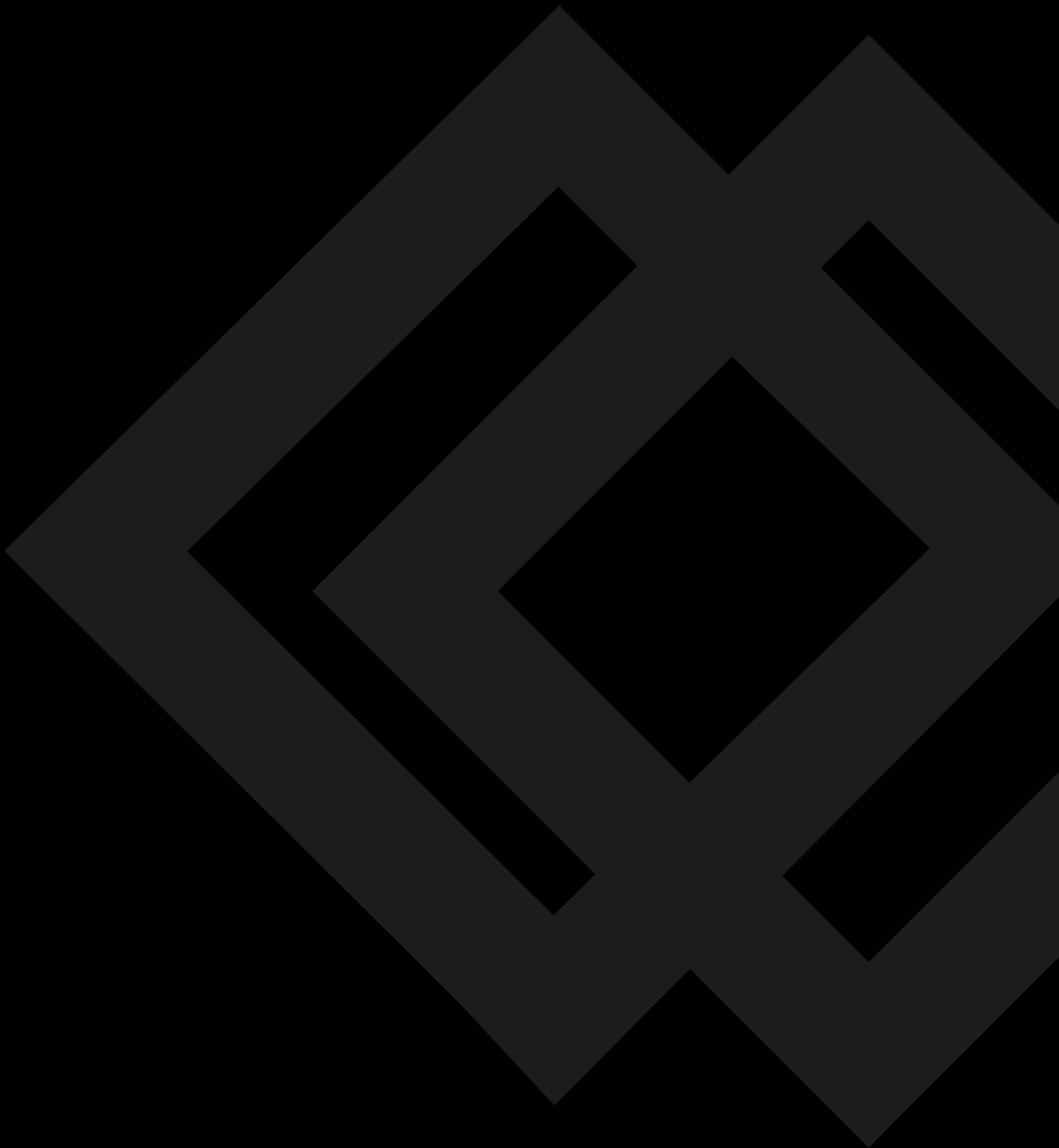


달지, 뽕뽕 등 고전적 탄막 회피 게임이 모티브

랜덤으로 생성되는 탄환을 피하며
최대한 오래 버티는 게임

2. 인터페이스

1. 게임설명



1. 게임설명

3. 각부의 기능

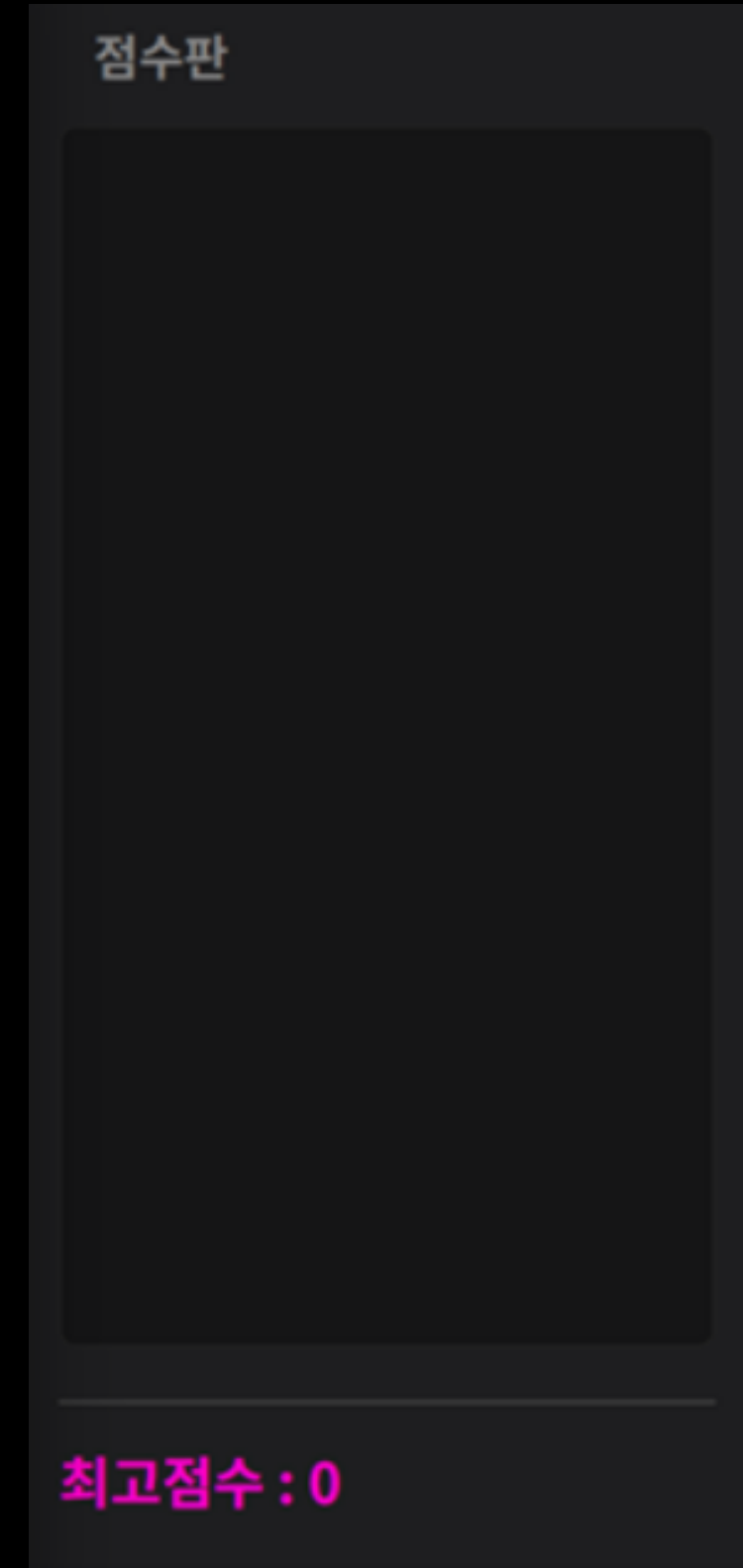


체력, 특수능력 게이지

점수 : 0

스테이지 : 1

현재 점수, 스테이지



게임 오버시 점수를
순차적으로 기록

최고 점수를 따로 표시함

1. 게임설명

4. 게임 진행



Space를 눌러 시작하고 방향키로 이동하여 진행

피격 혹은 아이템 획득시 점멸 및 무적시간 부여

Shift를 눌러 특수능력 사용 가능. 게이지 존재
모두 소진시 일정시간 재사용 불가능

R을 눌러 게임 리셋 가능



체력 회복, 상한 없음



특수능력 게이지 회복, 일정시간 최대치 유지



원형패턴 경고 표시

2. 주요기능

1. 페이드 인 & 아웃

```
@keyframes fadeInBullet { from { opacity: 0; } to { opacity: 1; } }
@keyframes fadeOutBullet { from { opacity: 1; } to { opacity: 0; } }
.bullet {
  position: absolute;
  color: ■yellow;
  opacity: 0;
  animation: fadeInBullet 0.5s ease-out forwards;
  box-shadow: 0 0 30px 3px ■yellow;
}
.bullet.out { animation: fadeOutBullet 1s ease-out forwards; }
```

객체 생성, 삭제시 투명도를 조절하여
페이드 인 & 아웃을 CSS로 구현.

당초 JS로 처리하였으나 객체가 많아질수록
로직 지연, 프레임 드랍 발생.
별도의 CSS로 따로 구현하여 해결함

또한 삭제 로직과 애니메이션을 분리하여
객체는 즉시 판정 제외하되 시각적으로는
서서히 사라지는 연출 부여.

2. 주요기능

2-1. 키입력

```
// 키 상태 관리 객체 (부드러운 이동, 대각 이동) 키,값 쌍
let keys = {};
// 누른 키 true로 함
window.onkeydown = (e) => {
    keys[e.keyCode] = true;
    if (e.keyCode == 32 && !isLock) startgame(); // 게임이 실행중이 아닐때만 실행
    if (e.keyCode == 82) reset();
    if (e.keyCode == 68) forceEnd();
};
// 떴을 키 false로 함
window.onkeyup = (e) => { keys[e.keyCode] = false; }
```

onkeydown / onkeyup 이벤트를 활용해
키 상태 true / false 를 객체에 키, 값 쌍으로 실시간으로 저장.

브라우저의 키 입력 반복 지연을 방지하여
부드러운 프레임 단위 이동을 구현

또한 동시 키 입력을 통한 대각 이동 구현

2. 주요기능

2-2. 이동 및 특수능력 구현

```
function updatePlayer() {
  const speed = 5 * timeScale;
  if (keys[37] && x > 0) x -= speed; //좌
  if (keys[39] && x < (p_limit - playerSize)) x += speed; //우
  if (keys[38] && y > 0) y -= speed; //상
  if (keys[40] && y < (p_limit - playerSize)) y += speed; //하

  if (keys[16] && gauge > 0 && !isCooldown) { //Shift
    timeScale = 0.3;
    gauge -= consumeRate;
    if (gauge < 0) {
      gauge = 0; // 0 이하 감소 방지
      isCooldown = true; // 게이지 소진시 쿨다운 적용
    }
    gamezone.style.background = "#112233"
  } else {
    timeScale = 1;
    if (gauge < maxGauge) {
      gauge += recoverRate;
      if (gauge >= 30 && isCooldown) { // 30 이상 회복시 쿨다운 해제
        isCooldown = false;
      }
      if (gauge > maxGauge) gauge = maxGauge; // 최대값 이상 증가 방지
    }
    gamezone.style.background = "black"
  }
  document.querySelector("#gauge-bar").style.width = gauge + "%";
  document.querySelector("#gauge-bar").style.background = isCooldown ? "red" : "cyan";

  // 좌표 값 전달
  player.style.left = x + "px";
  player.style.top = y + "px";
  // 좌표 그리기
  document.querySelector("#pos").innerHTML = `${Math.round(x)}, ${Math.round(y)}`;
}
```

상하좌우 방향키와 Shift 키 입력 이벤트

메인 함수 startgame()에서 초당 60 프레임 단위로 호출하여 사용

방향키 입력시 실시간 좌표값 변화

Shift키 입력시 timeScale을 변화시켜 슬로우모션 구현.
플레이어 및 탄환 속도에 일괄적으로 적용

배경 및 특수능력 게이지에 시각적 요소 추가

3. 탄환 생성자

2. 주요기능

<> 9_game.html > html > body > script > updatePlayer

```
function MovingObj(fixedX, fixedY, vX, vY, isPattern = false) {
  this.isDead = false;
  this.isWaiting = true;
  this.isPattern = isPattern;
  this.spawnTime = Date.now();
  this.kind = -1;

  // 매개변수 미정의시 기본값(랜덤) 지정
  this.x = (fixedX !== undefined) ? fixedX : nextRandomInteger(p_limit - 15);
  this.y = (fixedY !== undefined) ? fixedY : nextRandomInteger(p_limit - 15);
  this.vX = (vX !== undefined) ? vX : randomSpeed(6);
  this.vY = (vY !== undefined) ? vY : randomSpeed(6);

  this.body = document.createElement("div");
  this.body.innerHTML = '●';
  // this.body.className = "bullet";
  this.body.classList.add("bullet");
  this.body.style.fontSize = bulletSize + "px";
  this.body.style.width = bulletSize + "px";
  this.body.style.height = bulletSize + "px";
  this.body.style.left = this.x + "px";
  this.body.style.top = this.y + "px";
  if (this.isPattern) {
    this.body.style.color = "red";
  }
  gamezone.appendChild(this.body);

  // 생성 후 대기
  let timeoutId = setTimeout(() => {
    this.isWaiting = false;
  }, 1000);
  spawnTimers.push(timeoutId);
}
```

ex44_catchtext의 수정본

기본 탄환은 랜덤한 좌표 생성 후 랜덤한 속도로 이동함.

패턴 탄환은 매개변수를 받아와 각 패턴만의 독특한 움직임을 구현

탄환 생성 직후 피격을 방지하기 위해서 setTimeout으로 대기 시간 설정

2. 주요기능

4. 패턴 탄환

```
function spawnSpreadPattern(count) { // 방사탄환
  const sx = nextRandomInteger(p_limit - 20);
  const sy = nextRandomInteger(p_limit - 20);
  let speed = 3;
  let i = 0;
  let circleTimer = setInterval(function() {
    if (!isLock) { clearInterval(circleTimer); return; } //게임이 실행중이 아니면 바로 리턴
    const angle = (Math.PI * 2 / count) * i;
    const vX = Math.cos(angle) * speed;
    const vY = Math.sin(angle) * speed;
    MovingObjs.push(new MovingObj(sx, sy, vX, vY, true));
    i++;
    if (i >= count) {
      clearInterval(circleTimer);
    }
  }, 50);
  spawnTimers.push(circleTimer);
}
```

랜덤한 시작지점으로부터 사방으로 방향을 바꾸며 연속으로 발사되는 탄환

360도를 탄환 수로 나누어 간격을 구해 간격의 배수로 방향 설정

삼각함수를 이용해 x,y를 통한 대각 이동량(벡터)를 산정 후 이동에 부여

인터벌을 통해 순차적으로 생성 후 발사

2. 주요기능

5. 무적 시간 및 피격 로직

```
function setInvin() {
  isInvin = true;
  // 깜빡임 부여
  let cnt = 0;
  let blink = setInterval(() => {
    player.style.opacity = (cnt % 2 == 0) ? "0.1" : "0.7";
    cnt++;
  }, 100);
  let timeoutId = setTimeout(() => {
    clearInterval(blink);
    isInvin = false;
    player.style.opacity = "1";
  }, 1000);
  spawnTimers.push(timeoutId);
  spawnTimers.push(blink);
}

// 피격 로직
function hit(b) {
  // 플레이어 중심점
  const px = x + playerSize / 2;
  const py = y + playerSize / 2;
  // 탄환 중심점
  const bx = b.x + bulletSize / 2;
  const by = b.y + bulletSize / 2;

  // 두 중심점 사이의 거리. 삼각형의 빗변
  const dist = Math.sqrt((px - bx)**2 + (py - by)**2);
  // 두 중심점 사이의 거리가 (플레이어 반지름 + 탄환 반지름)보다 작으면 충돌
  return dist < (playerSize / 2 - 3) + (bulletSize / 2);
}
```

* 무적 시간 함수

생성, 피격 직후 연속 피격을 방지하기 위해서 무적 시간 부여
인터벌로 투명도를 조절하여 깜빡임 구현
isInvin 변수를 통해 충돌 로직을 비활성화

* 피격 로직 함수

플레이어의 중심점과 탄환의 중심점 사이의 거리를 비교하여 피격 여부 판단
피타고라스 정리를 이용하여 x, y 를 기준으로 중심점끼리의 거리(빗변)를 산출.
계산된 결과가 두 객체의 반지름 합보다 작을 경우 충돌 판정
아슬아슬한 효과를 내기 위해 실제보다 작은 히트박스로 관대한 피격 판정 설정

2. 주요기능

6-1. 메인 로직 전반

```
function startgame() {
  isLock = true;
  document.querySelector("#start-msg").style.display = "none";
  startTime = Date.now();
  setInvin();
  bulletSpawn();
  spawnRainPattern(1);
  game = setInterval(function() {
    updatePlayer(); // 매 프레임 플레이어 위치 업데이트

    score = Date.now() - startTime; // 현재시간-시작시간으로 점수 산정
    viewScore = Math.round(score / 100);
    document.querySelector("#score").innerHTML = `점수 : ${viewScore}<br> 스테이지 : ${1+currentLevel}`;

    // 3초당 레벨 상승
    let levelchk = Math.floor(score / 3000);
    if (levelchk > currentLevel) {
      currentLevel = levelchk;

      // 탄환 생성
      if (currentLevel % 5 == 0) spawnCirclePattern(currentLevel*1);
      else if (currentLevel % 2 == 0) spawnSpreadPattern(currentLevel*3);
      else bulletSpawn(currentLevel);

      if (currentLevel % 2 == 0) { itemSpawn(Math.floor(Math.random()*2)) }

      // 무작위 방향 패턴 생성. 1~9 중 1~4만 생성(40%)
      let dice = Math.floor(Math.random() * 10) + 1;
      spawnRainPattern(dice);
    }
  }, 1000);
}
```

isLock 상태 조절 변수. 연속 실행 방지
시작 시간과 종료시간 차이를 이용하여 스코어 산정

약 3초당 레벨 상승, 레벨에 따른 탄환 갯수 증가
레벨 상승마다 확률적으로 무작위 방향 RainPattern 발생

```
function spawnRainPattern(d) {
  if(d > 4) return;
  let num = 0;
  let rx = 0;
  let ry = 0;
  let vX = 0;
  let vY = 0;

  if (d == 1) { ry = -10, vY = 5; } //상
  else if (d == 2) { ry = 720; vY = -5; } //하
  else if (d == 3) { rx = -10, vX = 5; } //좌
  else if (d == 4) { rx = 720, vX = -5; } //우

  let spawnTimer = setInterval(()=> {
    MovingObjs.push(new MovingObj(rx,ry,vX,vY,true))
    num++;
    if (d == 1 || d == 2) { rx += 50;}
    else if(d == 3 || d == 4 ){ ry += 50;}
    if(num > 14) clearInterval(spawnTimer);
  },100);
  spawnTimers.push(spawnTimer);
}
```


2. 주요기능

6-1. 메인 로직 후반

```
for (let i = MovingObjs.length - 1; i >= 0; i--) {  
  let b = MovingObjs[i];  
  b.move();  
  if (!isInvin && !b.isWaiting && hit(b)) {  
    if (b.kind == 0) {  
      life += 2;  
      b.remove();  
    } else if (b.kind == 1) {  
      life++;  
      let boost = setInterval(()=>{  
        gauge = 100;  
        isCooldown = false;  
        document.querySelector("#gauge-bar").style.background = "gold";  
      }, 1000/60)  
  
      spawnTimers.push(boost);  
  
      let release = setTimeout(()=>{  
        document.querySelector("#gauge-bar").style.background = "cyan";  
        clearInterval(boost)  
      }, 2000)  
      b.remove();  
      spawnTimers.push(release);  
    }  
    life--;  
    lifeView();  
    setInvin();  
    if (life <= 0) gameEnd();  
  }  
}
```

생성된 객체들 관리.

생성된 모든 탄환과 아이템을 단일 배열(MovingObjs)에 저장하여 일괄 제어.
역순으로 순회하여 객체가 삭제됐을 경우에도 인덱스 밀림을 방지하여 꼬임방지.

무적상태인지, 생성 직후인지 판단하여 피격 로직 제어.
아이템 습득도 같이 관리하여 하나의 로직으로 처리

2. 주요기능

7. 리셋 함수

```
function reset() {  
  clearInterval(game);  
  //남아있는 모든 타이머 제거  
  spawnTimers.forEach(timerId => {  
    clearInterval(timerId);  
    clearTimeout(timerId);  
  });  
  spawnTimers = [];  
  isLock = false;  
  document.querySelector("#start-msg").style.display = "block";  
  // 모든 탄환 제거  
  [...MovingObjs].forEach(b => b.remove()); // Spread Operator  
  MovingObjs = []; // 배열 비우기  
  x = 345; y = 345;  
  life = 6;  
  currentLevel = 0;  
  gauge = 100;  
  isCooldown = false;  
  document.querySelector("#score").innerHTML = `점수 : 0<br> 스테이지 : 1`;  
  lifeView()  
  player.style.left = "345px";  
  player.style.top = "345px";  
  keys = {};  
}
```

reload() 사용시 점수판 기록이 초기화된 문제를 극복하기 위해
게임 영역만 초기화 하는 reset() 메서드 구현

reload() 미사용시
실행중인 타이머와 인터벌이 여전히 중첩실행되어 문제 발생 가능성 있음

모든 타이머와 인터벌을 배열에 담아 일괄적으로 삭제

3. 보완 및 느낀점

기존 예제를 수정해서 쉽게 가려고 했었는데 막상 시작하고 보니 오히려 지저분하게 코딩하게 된거 같아 아쉬움.

처음부터 구조를 설계하고나서 코딩했으면 더 수월 했을 듯 하다.
초기 설계의 중요성을 더 뼈저리게 느낌.

생성 로직 같은 공통적인 부분은 통합한 뒤 상속을 통해 구현하거나
메인 로직부분을 좀 더 기능별로 구분하는 객체지향적인 접근이 더 필요하다고 느꼈음.

고정된 크기를 가지고 있어서 가변 해상도에 대응 가능하게 반응형으로 설계하고
탄환과 아이템을 더 분리해서 코딩하지 못한게 아쉬움.

CSS부분은 대부분이 AI 의존적이어서 여기는 따로 좀 더 보완할 필요성을 느낌.

